

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING

Khwopa College Of Engineering

Libali, Bhaktapur

Department of Computer Engineering



A PROJECT REPORT ON

SECURE DOCUMENT MANAGEMENT SYSTEM
USING STEGANOGRAPHY

Team: ByteForge

Submitted in partial fulfillment of the requirements for the course

OBJECT ORIENTED PROGRAMMING
BACHELOR OF COMPUTER ENGINEERING

Submitted by

Aayush Khatri KCE082BCT002

Binit Maharjan KCE082BCT012

Biraj Khatri KCE082BCT013

Riyan Dhaubhadel KCE082BCT032

Khwopa College Of Engineering

Libali, Bhaktapur

2083/ /

CERTIFICATE OF APPROVAL

The undersigned certify that the project entitled “SECURE DOCUMENT MANAGEMENT SYSTEM USING STEGANOGRAPHY” submitted by Aayush Khatri (KCE082BCT002), Binit Maharjan (KCE082BCT012), Biraj Khatri (KCE082BCT013) and Riyan Dhaubhadel (KCE082BCT032) to the Department of Computer Engineering in partial fulfillment of the requirements of the Object Oriented Programming course for the degree of Bachelor of Computer Engineering, has been carried out under special supervision and within the time frame prescribed by the syllabus.

.....

Er. Dinesh Man Gothe

Head of Department

Department of Computer Engineering,

Khwopa College of Engineering

Copyright

The authors have agreed that the library, Khwopa College of Engineering may make this report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisor who supervised the project work recorded herein, or, in their absence, by the Head of the Department of Computer Engineering.

It is understood that recognition will be given to the authors of this report and to the Department of Computer Engineering, Khwopa College of Engineering, in any use of the material contained in this report. Copying, publication, or other use of this report for financial gain without the approval of the department and the authors' written permission is prohibited.

Requests for permission to copy or make any other use of the material in this report, in whole or in part, should be addressed to:

Head of Department

Department of Computer Engineering

Khwopa College of Engineering (KhCE)

Libali,

Bhaktapur, Nepal.

Acknowledgement

We would like to express our sincere gratitude to Er. Dinesh Man Gothe for his invaluable guidance, continuous encouragement, and unwavering support throughout the preparation and completion of this report. His valuable suggestions and mentorship played a significant role in guiding the development of our project.

We would also like to extend our heartfelt appreciation to all our teachers for their valuable assistance, guidance, and support throughout this project. Their knowledge and encouragement greatly contributed to our learning and progress.

Furthermore, we would like to thank our fellow classmates for their cooperation, discussions, and valuable perspectives. Their suggestions and interactions helped us gain different insights and contributed to improving the quality of our work.

Finally, we would like to acknowledge everyone who contributed directly or indirectly to the successful completion of this report and project. Their support, encouragement, and cooperation created a positive environment for learning, development, and innovation.

We are truly grateful to everyone who has been a part of this journey and contributed to the successful completion of our project.

Submitted by:

Aayush Khatri KCE082BCT002

Binit Maharjan KCE082BCT012

Biraj Khatri KCE082BCT013

Riyan Dhaubhadel KCE082BCT032

Abstract

The Secure Document Management System Using Steganography (**Steganshield**) is a desktop application developed in Python that addresses the problem of securing sensitive documents through multiple, complementary layers of protection rather than relying on a single technique. The system combines user authentication with salted password hashing, symmetric encryption of stored document content, an image-based steganography module that conceals secret messages using the Least Significant Bit (LSB) technique, and a set of password-security utilities including a strength checker, password generator, and hash generator. A menu-driven command-line interface was developed first to validate the core logic of each module, after which a graphical user interface was built using PyQt5, organizing the system's features into a tabbed Documents, Steganography, Password Tools, and Report workspace. The project demonstrates practical, layered data security: authentication controls who may access the system, encryption protects document content at rest, and steganography conceals the very existence of a hidden message inside an ordinary image file. The completed system allows a user to register and log in, manage an encrypted document collection with search and update functionality, hide and extract secret text within PNG images, evaluate and generate strong passwords, and view a summary report of system usage — all within a single, cohesive, locally-run application.

Table of Contents

Certificate of Approval.....	ii
Copyright.....	iii
Acknowledgement.....	iv
Abstract.....	v
List of Figures.....	ix
List of Tables.....	x
List of Abbreviations	xi
Chapter 1	1
Introduction.....	1
1.1 Background	1
1.2 Problem Statement.....	1
1.3 Objectives.....	2
1.4 Scope.....	2
1.5 Limitations.....	2
Chapter 2	3
Literature Review	3
2.1 Existing Approaches to Data Security	3
2.2 Image-Based Steganography and the LSB Technique	3
2.3 Password Security Practices	3
2.4 Why This Project Combines Multiple Techniques	4
Chapter 3	5
Requirement Analysis.....	5
3.1 Tools Used (Hardware and Software).....	5
3.2 Functional Requirements	5
3.3 Non-Functional Requirements.....	6
3.3.1 Reliability.....	6
3.3.2 Performance	6
3.3.3 Maintainability.....	6
3.3.4 Usability	6
3.3.5 Portability	6
3.3.6 Security	6
3.4 Feasibility Study.....	7
3.4.1 Economic Feasibility	7
3.4.2 Technical Feasibility	7

3.4.3 Operational Feasibility	7
Chapter 4	8
System Design and Architecture.....	8
4.1 Use Case Diagram	8
4.2 Class / Module Diagram	9
4.3 Sequence Diagram	9
4.4 Block Diagram / System Architecture.....	10
Chapter 5	11
Methodology	11
5.1 Software Development Approach.....	11
5.1.1 Stage 1 — Steganography Module	11
5.1.2 Stage 2 — Password Tools Module	11
5.1.3 Stage 3 — Encryption Module.....	11
5.1.4 Stage 4 — Authentication Module.....	11
5.1.5 Stage 5 — Document Management Module	11
5.1.6 Stage 6 — Command-Line Integration.....	11
5.1.7 Stage 7 — Graphical User Interface	11
5.2 Object-Oriented Design.....	12
Encapsulation	12
Abstraction	12
Modularity	12
Menu-Driven Programming	12
5.3 Data / File Handling.....	13
5.4 Module Description.....	13
5.4.1 Authentication Module (auth.py)	13
5.4.2 Document Management Module (documents.py)	13
5.4.3 Encryption Module (crypto_utils.py).....	13
5.4.4 Steganography Module (stego.py)	13
5.4.5 Password Tools Module (password_tools.py).....	13
5.5 Front-End Design.....	14
5.6 Overall Workflow	16
Chapter 6	17
Result and Discussion	17
6.1 Testing.....	17
6.2 Sample Sessions.....	17
6.3 Discussion.....	20
Chapter 7	20

7.1 Limitations.....	20
7.2 Future Enhancements.....	21
Appendix.....	22
Appendix A: Project File Structure	22
Appendix B: Sample Data Files	22
Appendix C: GitHub Repository	22

List of Figures

Figure 4.1 System Use Case Diagram

Figure 4.2 Class / Module Diagram

Figure 4.3 Sequence Diagram for the Add Document Flow

Figure 4.4 Block Diagram of the System Architecture

Figure 5.1 Login and Register Window

Figure 5.2 Main Window — Documents Tab

Figure 5.3 Main Window — Steganography Tab

Figure 5.4 Main Window — Password Tools Tab

List of Tables

Table 5.1 Data Files Used for Persistence

Table 6.1 Test Cases and Results

List of Abbreviations

CLI Command Line Interface

GUI Graphical User Interface

IDE Integrated Development Environment

JSON JavaScript Object Notation

LSB Least Significant Bit

OOP Object-Oriented Programming

PBKDF2 Password-Based Key Derivation Function 2

RGB Red Green Blue (colour model)

UI User Interface

UML Unified Modeling Language

Chapter 1

Introduction

1.1 Background

Digital documents routinely carry sensitive personal, financial, or confidential information, yet a great deal of everyday document storage relies on nothing more than an operating system's default file permissions. A single compromised account or an unencrypted backup can expose everything at once. Cryptography addresses part of this problem by making stored content unreadable without a key, but an encrypted file is still visibly identifiable as “something worth protecting,” which itself can draw unwanted attention.

Steganography offers a complementary approach: instead of scrambling a message, it hides the message's very existence inside an innocuous carrier, such as an ordinary image. Combined with encryption and conventional authentication, steganography allows a system to offer layered, defense-in-depth security rather than depending on any single mechanism.

This project, Secure Document Management System Using Steganography(**Steganshield**), was developed to explore this layered approach in practice. It combines a login and registration system, encrypted document storage, an image-based steganography module using the Least Significant Bit (LSB) technique, and a set of password-security utilities, all accessible through a graphical desktop interface built with PyQt5.

1.2 Problem Statement

Most introductory document-management projects demonstrate only one security concept in isolation — either a login system, or basic encryption, or a steganography demo — without showing how these techniques work together in a single, coherent application. This project addresses that gap by building a working system in which authentication, encryption, and steganography each play a distinct, complementary role: authentication controls who can access the system, encryption protects document content at rest, and steganography provides an additional channel for concealing small secrets, such as a passphrase or note, inside an image file where their presence is not obvious.

1.3 Objectives

General Objective

The main objective of this project is to develop a secure document management system in Python that demonstrates layered data security through authentication, encryption, and steganography.

Specific Objectives

- To develop a secure system for storing and managing documents.
- To implement steganography for hiding sensitive data inside images.
- To apply encryption techniques for data protection.
- To provide user-friendly features such as login, search, and record management.
- To implement password-security tools including a strength checker, generator, and hash generator.
- To build a menu-driven interface, followed by a graphical interface using PyQt5.

1.4 Scope

The scope of this project covers the development of a desktop application focused on secure document management. The current implementation includes user registration and login, encrypted document storage with add, view, search, update, and delete functionality, an image steganography module for hiding and extracting text messages, a set of password-security utilities, and a summary report screen. Data is stored locally using JSON files, and images are handled using the Pillow library. The system is developed entirely in Python, first as a command-line application and then as a PyQt5 graphical desktop application.

1.5 Limitations

As this is an academic prototype, the current implementation has several limitations. Data is stored in local JSON files rather than a dedicated database, which limits scalability for a large number of users or documents. The system currently runs on a single device rather than across a network. The steganography module works only with lossless image formats such as PNG, since lossy compression (e.g. JPEG) would destroy the hidden data. These limitations represent opportunities for further development, discussed in Chapter 7.

Chapter 2

Literature Review

2.1 Existing Approaches to Data Security

Data security in software systems is commonly approached through three broad techniques: authentication, cryptography, and steganography. Authentication systems verify the identity of a user before granting access, typically through a username and password checked against a securely stored hash. Cryptographic techniques transform readable data into an unreadable form using a secret key, protecting the content of a message but not concealing the fact that a protected message exists. Steganography takes a different approach: it hides a message inside another, unremarkable piece of data — commonly an image, audio file, or text — so that an observer is not aware a hidden message exists at all.

2.2 Image-Based Steganography and the LSB Technique

Among the various steganography techniques, Least Significant Bit (LSB) substitution is one of the most widely studied due to its simplicity and effectiveness for lossless image formats. Each pixel in a colour image is represented by Red, Green, and Blue (RGB) values, each stored as 8 bits. Because the least significant bit of each value contributes only marginally to the perceived colour, it can be replaced with a bit of secret data without producing a visually detectable change. This project applies the LSB technique to PNG images, using a fixed delimiter bit-pattern to mark the end of the hidden message during extraction.

2.3 Password Security Practices

Password-related vulnerabilities remain one of the most common causes of account compromise. Established best practice recommends storing password hashes rather than plaintext passwords, using a slow, salted hashing algorithm such as PBKDF2 to resist brute-force attacks, and encouraging users to choose passwords that combine length, character variety, and unpredictability. This project incorporates these practices directly: passwords are hashed using PBKDF2 with a per-user salt, and a strength-checking utility gives users immediate feedback when registering.

2.4 Why This Project Combines Multiple Techniques

Rather than presenting steganography as an isolated demonstration, this project situates it within a broader, layered security model, alongside authentication and encryption. This design choice reflects a widely recognised principle in security engineering known as defense in depth: no single technique should be relied upon exclusively, since each has different strengths and each protects against a different class of threat. This literature review therefore frames the project's design around three complementary layers — authentication, encryption, and steganography — rather than treating steganography as the system's only security feature.

Chapter 3

Requirement Analysis

3.1 Tools Used (Hardware and Software)

The hardware used for development was standard personal computing hardware (laptops), which was sufficient given the project does not require GPU acceleration or large-scale data processing. The following software and tools were used:

- Python 3 (interpreter and standard library)
- Pillow (image processing for steganography)
- cryptography library (Fernet symmetric encryption)
- PyQt5 (graphical user interface)
- Visual Studio Code (development and editing)
- Git and GitHub (version control)
- JSON files (.json) as a lightweight, file-based data store

3.2 Functional Requirements

The system shall provide the following functionality:

- Allow a new user to register with a username and password.
- Allow a registered user to log in using their username and password.
- Allow a logged-in user to add, view, search, update, and delete documents.
- Encrypt document content before it is stored, and decrypt it on retrieval.
- Allow a user to hide a secret text message inside a PNG image.
- Allow a user to extract a hidden message from an encoded PNG image.
- Allow a user to check the strength of a password.
- Allow a user to generate a random strong password.
- Allow a user to generate a hash of any given text.
- Allow a user to view a summary report of users and documents.

3.3 Non-Functional Requirements

3.3.1 Reliability

The system should run without crashing under normal, expected input, and file operations should fail gracefully — for example, an empty documents file simply produces an empty listing rather than an error.

3.3.2 Performance

Given the lightweight, file-based nature of the prototype, the system should respond to all menu and GUI operations effectively instantly, since operations are simple reads and writes over small JSON files and images.

3.3.3 Maintainability

Each feature area is implemented as an independent Python module (auth, documents, stego, crypto_utils, password_tools) with a small, focused set of functions, so that a module can be modified, extended, or replaced with minimal impact on the rest of the codebase.

3.3.4 Usability

The command-line interface uses clearly numbered menus, and the PyQt5 graphical interface organizes features into labelled tabs with forms, tables, and clear feedback dialogs, so a first-time user can navigate the system without prior instructions.

3.3.5 Portability

The system is written in standard Python 3 using cross-platform libraries (Pillow, cryptography, PyQt5), and is expected to run on Windows, Linux, or macOS without modification.

3.3.6 Security

Passwords are never stored in plaintext; they are hashed using PBKDF2 with a per-user salt. Document content is encrypted at rest using Fernet symmetric encryption. The steganography module provides an additional layer of concealment for small secrets embedded in images.

3.4 Feasibility Study

3.4.1 Economic Feasibility

No paid infrastructure, licensing, or cloud services are required for the prototype. Development relies entirely on free, open-source tools (Python, PyQt5, Git) and the team's own computers, making the project economically feasible at zero direct cost.

3.4.2 Technical Feasibility

The prototype relies on well-established, mature technologies — Python, Pillow, the cryptography library, and PyQt5 — all of which are well documented and within the technical capability of the team, keeping the technical requirements modest and achievable.

3.4.3 Operational Feasibility

The application requires no special training beyond following the on-screen menus and tabbed interface. The entire system runs locally on a single laptop without any deployment infrastructure, which keeps the operational overhead of running and evaluating the project minimal.

Chapter 4

System Design and Architecture

The system is organized around a PyQt5 graphical front end that calls into a set of independent Python modules, each responsible for one functional area. This chapter presents the use case, class/module, sequence, and architectural (block) views of the system.

4.1 Use Case Diagram

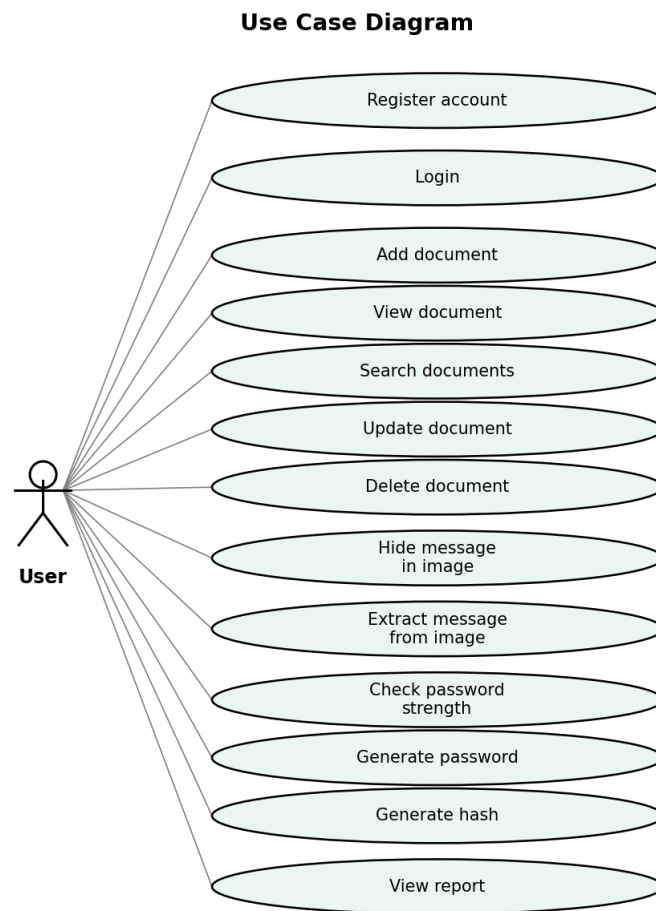


Figure 4.1: System Use Case Diagram

Figure 4.1 shows the single actor — a User — who interacts with the system. A user can register, log in, and then manage documents (add, view, search, update, delete), use the steganography features (hide and extract a message in an image), use the password tools (check strength, generate a password, generate a hash), and view the system report.

4.2 Class / Module Diagram

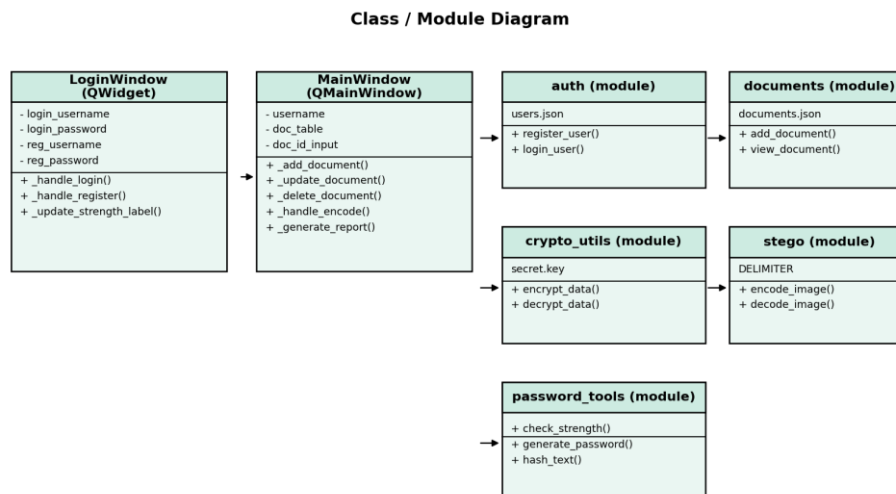


Figure 4.2: Class / Module Diagram

Figure 4.2 shows the structure of the system. LoginWindow and MainWindow are PyQt5 classes (inheriting QWidget and QMainWindow respectively) that implement the graphical interface. They call into five independent Python modules — auth, documents, crypto_utils, stego, and password_tools — each encapsulating its own private data file and exposing a small, purposeful set of public functions, following the single-responsibility principle.

4.3 Sequence Diagram

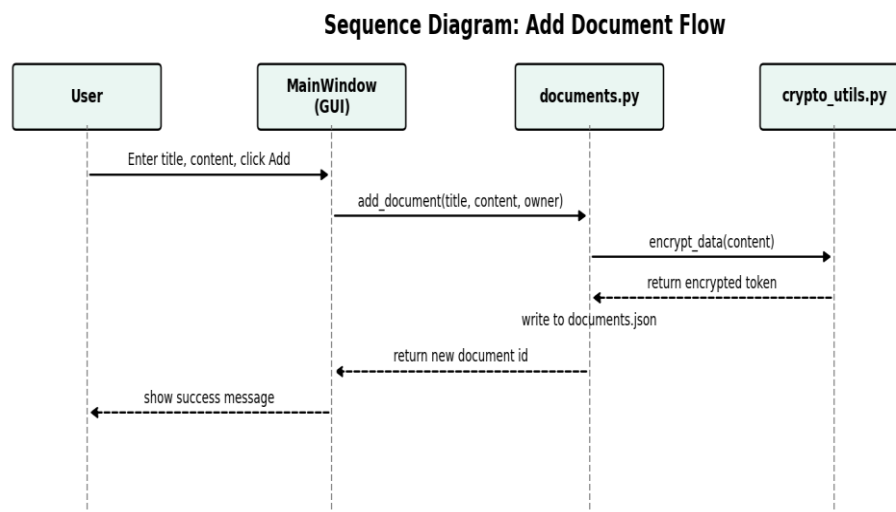


Figure 4.3: Sequence Diagram for the Add Document Flow

Figure 4.3 illustrates the interaction that occurs when a user adds a new document. The MainWindow receives the user's form input and calls add_document() in the documents module, which in turn calls encrypt_data() in the crypto_utils module before writing the

encrypted record to documents.json. A similar request-response pattern is followed by every other document operation, with the relevant function substituted for add_document().

4.4 Block Diagram / System Architecture

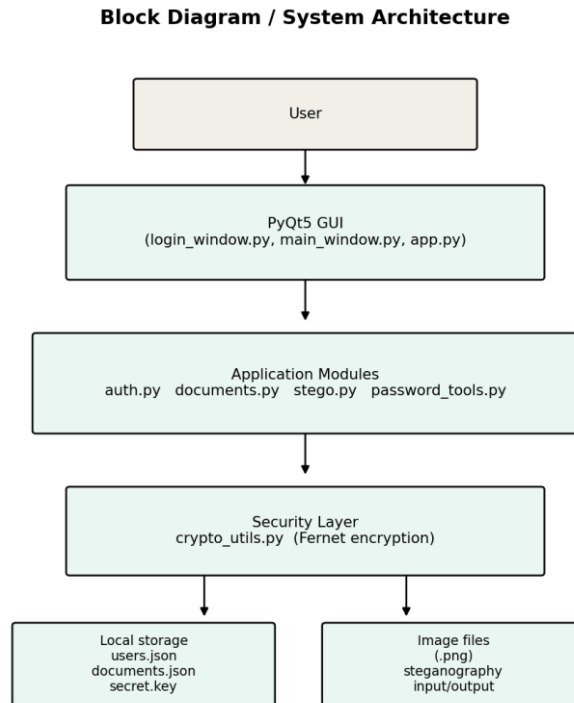


Figure 4.4: Block Diagram of the System Architecture

Figure 4.4 summarizes the overall architecture. The user interacts with the PyQt5 graphical interface, which delegates to the application modules (auth, documents, stego, password_tools). Document operations pass through the security layer (crypto_utils) before reaching local storage, which consists of JSON files for user and document records and PNG image files for steganography input and output.

Chapter 5

Methodology

5.1 Software Development Approach

A prototyping approach was adopted for the development of this system. This approach allowed the team to gradually design, implement, test, and improve different components rather than developing the complete application at once. Each stage focused on implementing specific functionality while allowing continuous evaluation and refinement.

5.1.1 Stage 1 — Steganography Module

Implemented the core LSB encode and decode functions using the Pillow library, along with a simple command-line menu for testing.

5.1.2 Stage 2 — Password Tools Module

Implemented password strength checking, random password generation, and hash generation using the hashlib and string libraries.

5.1.3 Stage 3 — Encryption Module

Implemented key generation, encryption, and decryption using the Fernet symmetric encryption scheme from the cryptography library.

5.1.4 Stage 4 — Authentication Module

Implemented user registration and login, storing salted PBKDF2 password hashes in a JSON file rather than plaintext passwords.

5.1.5 Stage 5 — Document Management Module

Implemented add, view, search, update, and delete operations for documents, with content encrypted before being written to disk and decrypted only on retrieval.

5.1.6 Stage 6 — Command-Line Integration

Integrated all modules into a single menu-driven command-line application (main.py) to verify the complete flow before building a graphical interface.

5.1.7 Stage 7 — Graphical User Interface

Rebuilt the application as a PyQt5 desktop application, with a login/register window and a tabbed main window covering Documents, Steganography, Password Tools, and Report, styled with a custom stylesheet.

5.2 Object-Oriented Design

The graphical front end of the system was designed using Object-Oriented Programming principles, while the underlying application logic was organized into cohesive functional modules — a hybrid design common in Python applications, and one that still demonstrates the core OOP concepts covered in the course.

Encapsulation

The LoginWindow and MainWindow classes encapsulate their own internal state (input fields, tables, form data) as private instance attributes, exposing only the methods needed to respond to user actions. Similarly, each module keeps its data file path and internal helper logic private, exposing only a small public function interface (for example, `encrypt_data()` and `decrypt_data()` in `crypto_utils`).

Abstraction

Each module hides its internal implementation behind simple, meaningful function names. The GUI simply calls functions such as `add_document()`, `encode_image()`, or `check_strength()` without needing to know how encryption, file storage, or bit manipulation are carried out internally.

Modularity

The system is divided into five independent modules — `auth`, `documents`, `crypto_utils`, `stego`, and `password_tools` — plus the GUI layer. Each module can be tested, modified, or replaced independently, which was demonstrated in practice when the command-line interface was replaced by the PyQt5 interface without changing any of the underlying module logic.

Menu-Driven Programming

Both the command-line prototype and the graphical interface follow a menu-driven design, presenting the user with a clear set of numbered or tabbed choices at every stage rather than requiring memorized commands.

5.3 Data / File Handling

The system uses JSON files for structured data and PNG image files for steganography input and output, avoiding the need for a full database server for this prototype.

File	Purpose	Format
users.json	Stores usernames, password hashes, and salts	JSON
documents.json	Stores document records with encrypted content	JSON
secret.key	Stores the Fernet encryption key	Binary
encoded.png	Output image containing a hidden message	PNG image

Table 5.1: Data Files Used for Persistence

5.4 Module Description

5.4.1 Authentication Module (auth.py)

Handles user registration and login. Passwords are hashed using PBKDF2 with a randomly generated per-user salt before being stored, and are never stored in plaintext.

5.4.2 Document Management Module (documents.py)

Provides add, view, search, update, and delete operations for documents. Document content is encrypted before being written to documents.json and decrypted only when a document is viewed.

5.4.3 Encryption Module (crypto_utils.py)

Provides key generation and Fernet-based symmetric encryption and decryption, used by the document management module to protect content at rest.

5.4.4 Steganography Module (stego.py)

Implements the LSB technique to hide a text message inside a PNG image and extract it again, using a fixed delimiter bit-pattern to mark the end of the hidden message.

5.4.5 Password Tools Module (password_tools.py)

Provides a password strength checker, a random strong password generator, and a general-purpose text hash generator supporting SHA-256, SHA-512, and MD5.

5.5 Front-End Design

The graphical interface was built using PyQt5, organized around a LoginWindow and a tabbed MainWindow. A custom stylesheet (style.qss) was applied to give the application a consistent, modern flat design with a teal accent colour, rounded corners, and clear visual feedback on hover and focus states. The Documents tab combines a form for adding and editing records with a live table view; the Steganography tab provides file browsers for selecting images to encode or decode; the Password Tools tab combines a live strength checker, a password generator, and a hash generator; and the Report tab displays a live summary of system usage.

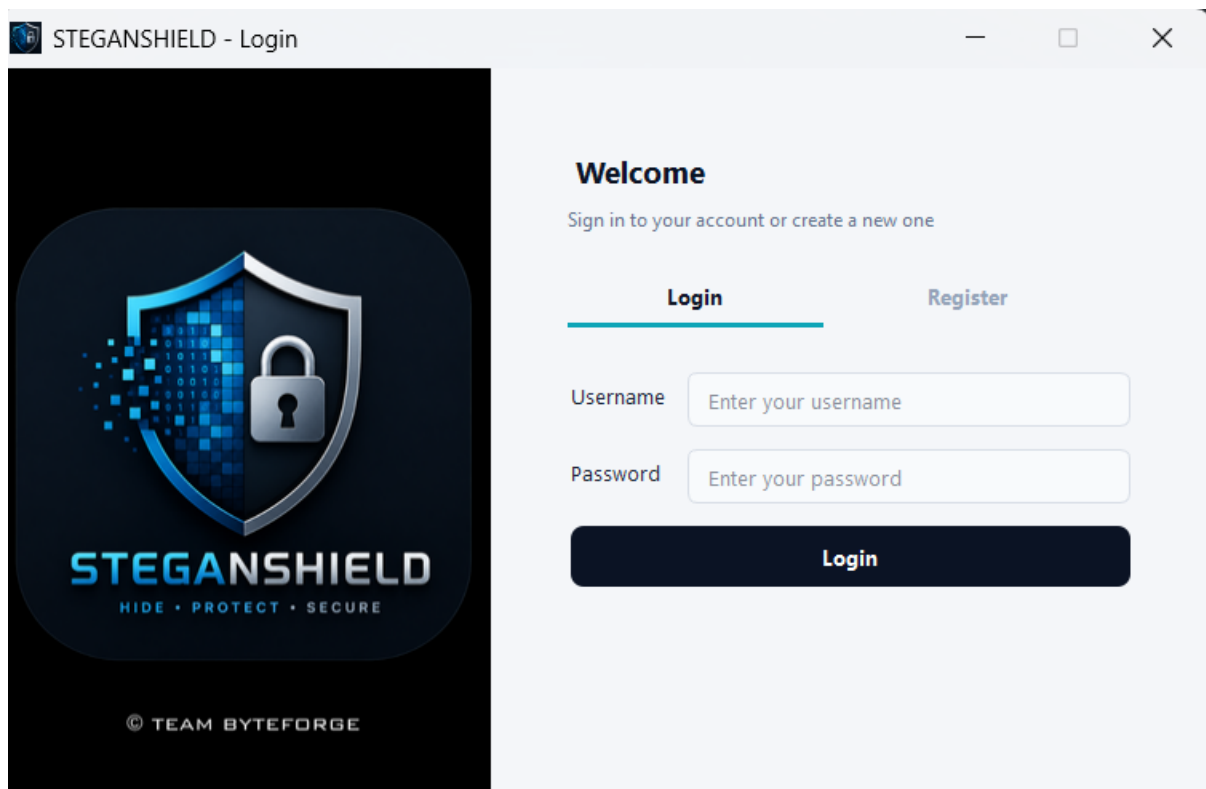


Figure 5.1: Login and Register Window]

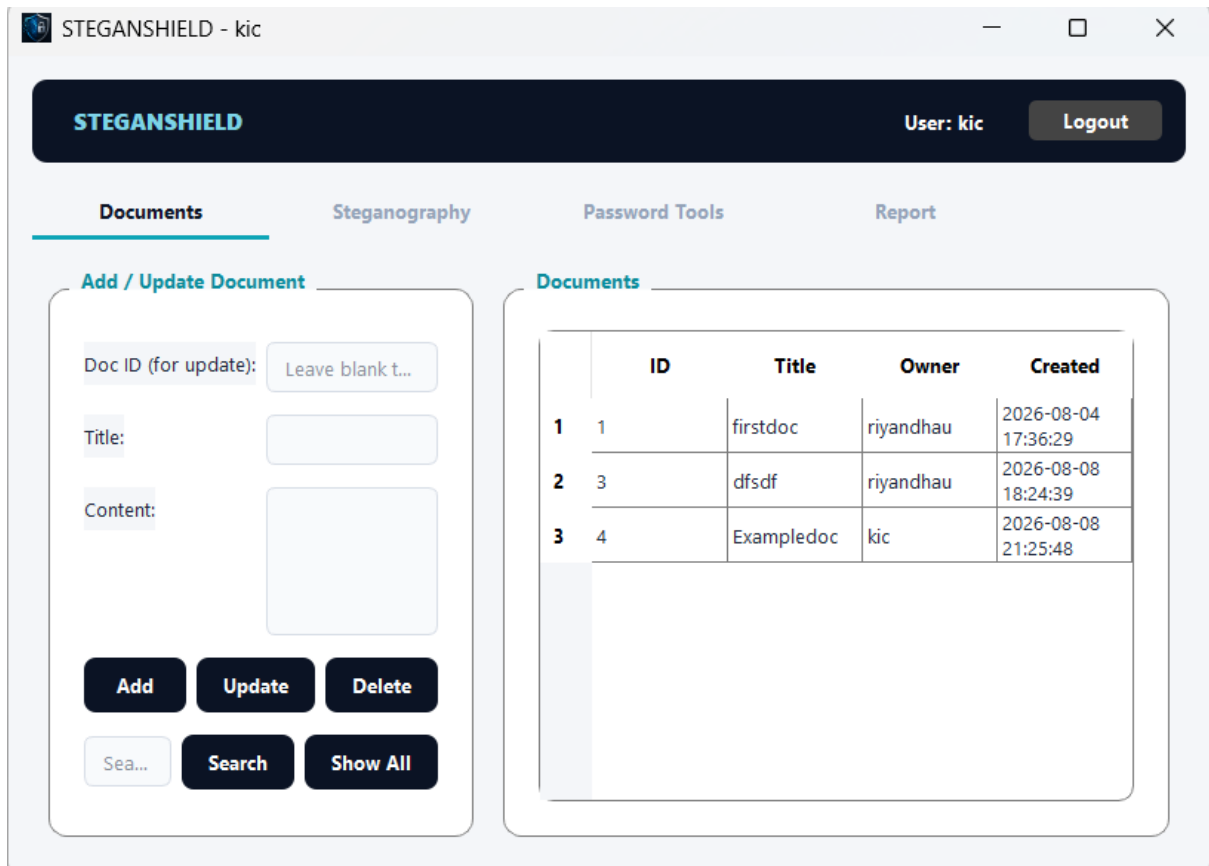


Figure 5.2: Main Window — Documents Tab

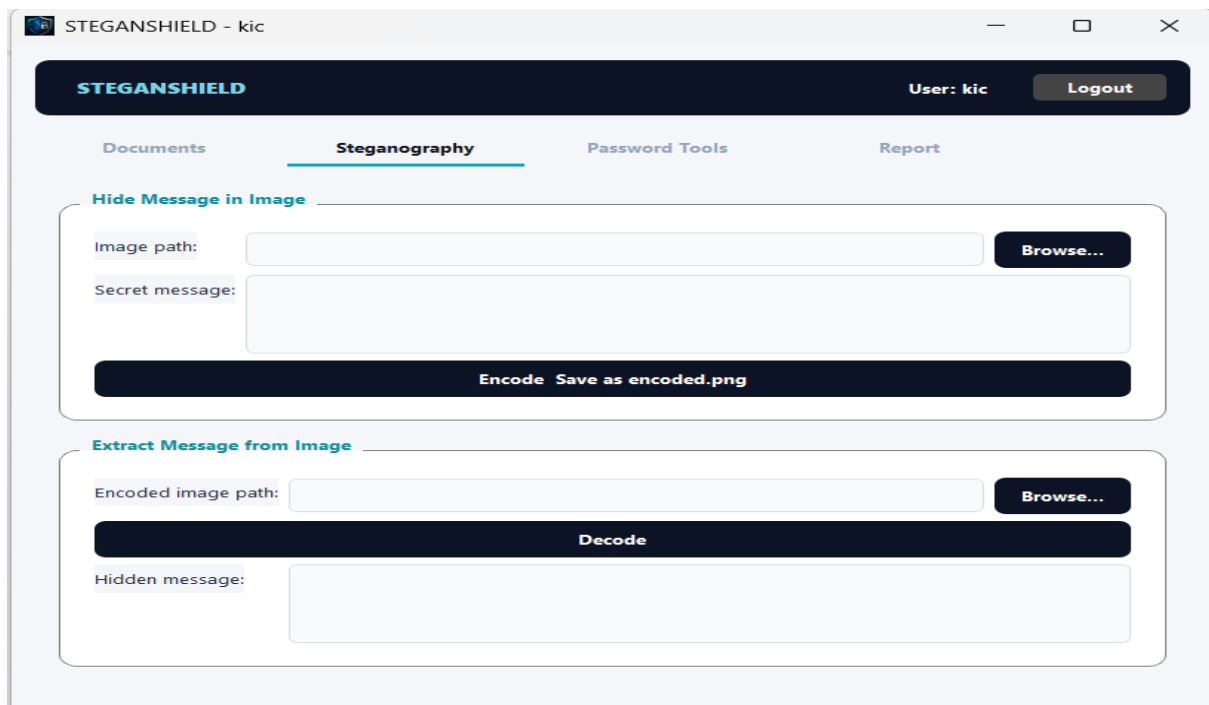


Figure 5.3: Main Window — Steganography Tab

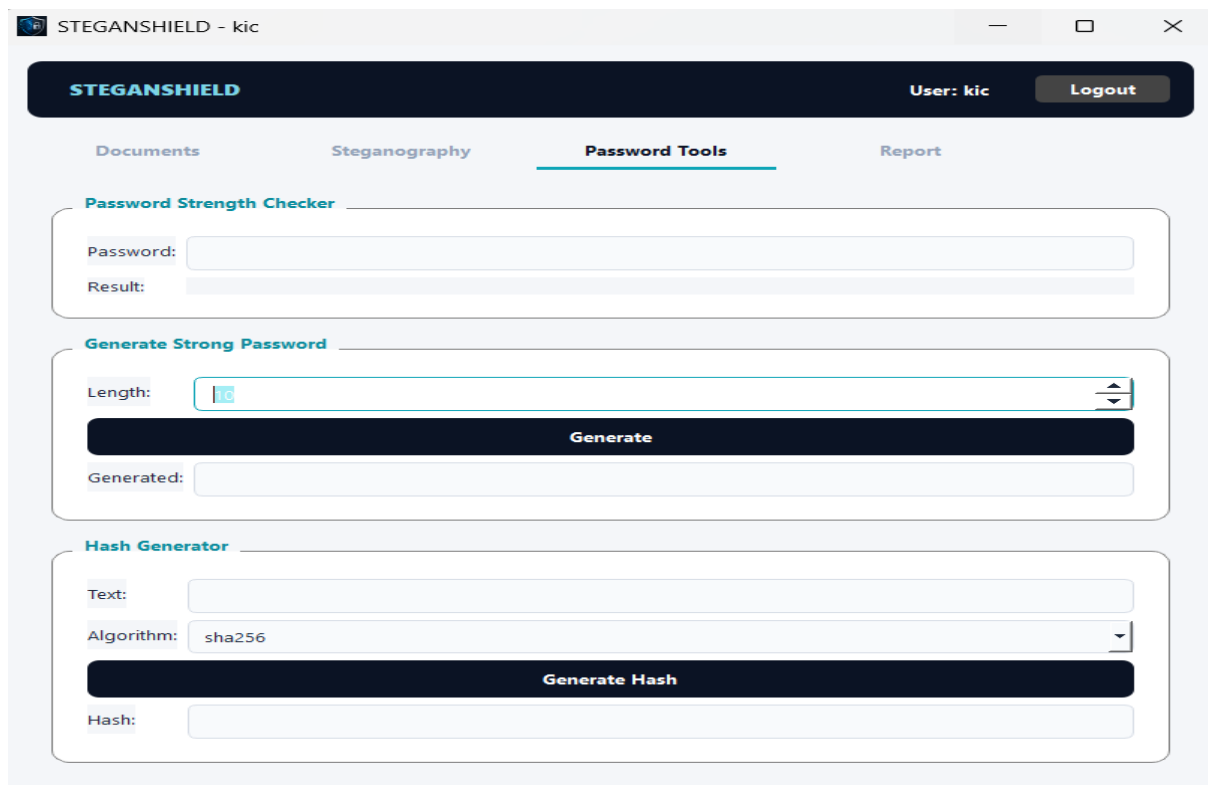


Figure 5.4: Main Window — Password Tools Tab

5.6 Overall Workflow

A user launches the application, registers or logs in, and is then presented with the main menu. From there, the user may freely move between the Documents, Steganography, Password Tools, and Report tabs before logging out, which returns the application to the login screen. The Password Tools and Steganography features operate entirely on the local machine and do not depend on the user being logged in to a specific document set, while the Documents and Report tabs are tied to the logged-in user's session.

Chapter 6

Result and Discussion

6.1 Testing

Each module was tested individually before integration, followed by end-to-end testing of the complete GUI workflow.

Test Case	Expected Result	Status
Register a new user	User is added to users.json with a hashed password	Pass
Login with correct credentials	Access granted to main window	Pass
Login with incorrect password	Access denied, error shown	Pass
Add a document	Document encrypted and stored, visible in list	Pass
View a document	Content correctly decrypted and displayed	Pass
Search documents by title	Matching documents returned	Pass
Hide a message in an image	encoded.png produced, image visually unchanged	Pass
Extract a message from an image	Original hidden message correctly recovered	Pass
Check password strength	Correct rating and missing-criteria feedback shown	Pass
Generate a password	Password meeting all strength criteria produced	Pass

Table 6.1: Test Cases and Results

6.2 Sample Sessions

```
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\auth.py"
1. Register
2. Login
3. Delete User
4. List Users
Enter choice: 2
Username: kic
Password: Birajkc@1
[+] Login successful. Welcome, kic!
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc>
```

Figure 6.1: Login

```
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\documents.py"
1. Add Document
2. View Document
3. List All Documents
4. Search Documents
5. Update Document
6. Delete Document
Enter choice: 1
Title: example
Content: example document
Owner (username): kic
[+] Document added with ID: 5
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc>
```

Figure 6.2: Add Document

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\documents.py"
1. Add Document
2. View Document
3. List All Documents
4. Search Documents
5. Update Document
6. Delete Document
Enter choice: 2
Document ID: 5
[+] Title: example
    Owner: kic
    Created: 2026-08-09 15:01:41
    Modified: 2026-08-09 15:01:41
    Content: example document
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc>

```

Figure 6.3: View Document

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\documents.py"
1. Add Document
2. View Document
3. List All Documents
4. Search Documents
5. Update Document
6. Delete Document
Enter choice: 3
[+] All documents:
    [1] firstdoc (owner: riyandhau, created: 2026-08-04 17:36:29)
    [3] df sdf (owner: riyandhau, created: 2026-08-08 18:24:39)
    [4] Exampledoc (owner: kic, created: 2026-08-08 21:25:48)
    [5] example (owner: kic, created: 2026-08-09 15:01:41)
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc>

```

Figure 6.4: List All Documents

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\documents.py"
1. Add Document
2. View Document
3. List All Documents
4. Search Documents
5. Update Document
6. Delete Document
Enter choice: 4
Search keyword: example
[+] Found 2 match(es):
    [4] Exampledoc (owner: kic)
    [5] example (owner: kic)
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc>

```

Figure 6.5: Search Documents

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\documents.py"
1. Add Document
2. View Document
3. List All Documents
4. Search Documents
5. Update Document
6. Delete Document
Enter choice: 5
Document ID to update: 5
New title (leave blank to keep unchanged): exmaplechanged
New content (leave blank to keep unchanged): exmaple doc got changed
[+] Document updated successfully.
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc>

```

Figure 6.6: Update Document

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\documents.py"
1. Add Document
2. View Document
3. List All Documents
4. Search Documents
5. Update Document
6. Delete Document
Enter choice: 6
Document ID to delete: 5
[+] Document deleted successfully.
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc>

```

Figure 6.7: Delete Documents

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\stego.py"
1. Encode
2. Decode
Enter choice: 1
Enter image path: C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\example.png
Enter secret message: encoded message is hello
c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\stego.py:16: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattened_data instead.
pixels = list(img.getdata())
[+] Message successfully encoded into 'encoded.png'

```

Figure 6.8: Encode

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\stego.py"
1. Encode
2. Decode
Enter choice: 2
Enter encoded image path: C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\encoded.png
c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\stego.py:56: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattened_data instead.
pixels = list(img.getdata())
[+] Hidden message: encoded message is hello

```

Figure 6.9: Decode

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\password_tools.py"
1. Check Password Strength
2. Generate Strong Password
3. Generate Hash of Text
Enter choice: 1
Enter password to check: Kic@12345
[+] Strength: Strong

```

Figure 6.10: Check Password Strength

```

PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\password_tools.py"
1. Check Password Strength
2. Generate Strong Password
3. Generate Hash of Text
Enter choice: 2
Enter desired length (default 12): 15
[+] Generated password: YE(>0v|T\;V6"Gi

```

Figure 6.11: Generate Strong Password

```
PS C:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc> python -u "c:\Users\LOQ\OneDrive\Desktop\Secfinal\securedoc\password_tools.py"
1. Check Password Strength
2. Generate Strong Password
3. Generate Hash of Text
Enter choice: 3
Enter text to hash: example
Algorithm (sha256/sha512/md5) [default sha256]: md5
[+] MD5 hash: 1a79a4d60de6718e8e5b326e338ae533
```

Figure 6.12: Generate Hash of Text

6.3 Discussion

Testing confirmed that the three security layers work correctly both individually and together. Authentication successfully prevents unauthorized access to the document management features. Encryption ensures that document content stored in documents.json is unreadable without the correct key, verified by inspecting the raw file contents directly. The steganography module successfully hid and recovered text messages of varying length without any visually detectable change to the carrier image, confirmed by comparing the original and encoded images side by side. The password tools module gave consistent, accurate strength ratings across a range of test passwords, reinforcing good password hygiene at the point of registration.

One practical limitation observed during testing was that the steganography module's capacity is directly tied to image size — larger secret messages require a proportionally larger carrier image, and the system correctly raises an error rather than silently truncating the message when capacity is exceeded.

Chapter 7

7.1 Limitations

- Data is stored in local JSON files rather than a dedicated database, which limits scalability.
- The system currently runs on a single device; it does not yet support multiple users accessing shared documents across a network.
- Steganography works only with lossless image formats (PNG); lossy formats such as JPEG would destroy the hidden data.
- The hidden message capacity is limited by the size of the carrier image.
- The LSB technique, while effective against casual visual inspection, is not resistant to dedicated steganalysis techniques.

7.2 Future Enhancements

- Replace JSON file storage with a proper database (e.g. SQLite or PostgreSQL) to support larger datasets.
- Add a client-server architecture so documents can be securely shared and accessed across multiple devices.
- Support additional steganography techniques and carrier formats, such as audio-based steganography.
- Add role-based access control, allowing different permission levels for different users.
- Add two-factor authentication to further strengthen the login system.
- Package the application as a standalone installable executable for easier distribution.

Appendix

Appendix A: Project File Structure

app.py — Application entry point; launches the login window and main window

login_window.py — PyQt5 login and registration window

main_window.py — PyQt5 main window with Documents, Steganography, Password Tools, and Report tabs

auth.py — User registration and login (PBKDF2 password hashing)

documents.py — Document CRUD operations with encrypted content

crypto_utils.py — Fernet symmetric encryption and decryption

stego.py — LSB steganography encode and decode functions

password_tools.py — Password strength checker, generator, and hash generator

style.qss — Application-wide stylesheet for the PyQt5 interface

data/ — Folder containing users.json, documents.json, and secret.key

Appendix B: Sample Data Files

Example structure of a record in users.json:

```
{ "alice": { "password_hash": "<hex string>", "salt": "<random salt>" } }
```

Example structure of a record in documents.json:

```
{ "1": { "title": "Notes", "content": "<encrypted token>", "owner": "alice", "date_created":  
"2082-01-01 10:00:00", "date_modified": "2082-01-01 10:00:00" } }
```

Appendix C: GitHub Repository

[<https://github.com/RiyanDhau/securedoc.git>.]